

Beginner-C Bootcamp 6주차 과제

20215789 산업보안학과 김문정

Challenge 1. 포인터 기초 실습

```
#include <stdio.h>

int main() {
    int num = 10;

    // num을 가리키는 포인터
    int *ptr = &num;

    printf("=== 변수 정보 ===\n");
    printf("num의 값: %d\n", num);
    printf("num의 주소: %p\n", &num);

    printf("\n=== 포인터 정보 ===\n");
    printf("ptr이 저장한 값(주소): %p\n", ptr);
    printf("ptr 자신의 주소: %p\n", &ptr);
    printf("ptr이 가리키는 값: %d\n", *ptr);

    // 포인터를 통해 값 변경
    *ptr = 50;

    printf("\n=== 값 변경 후 ===\n");
    printf("num의 값: %d\n", num);
    printf("*ptr의 값: %d\n", *ptr);
}
```

```
user@user-VirtualBox:~$ vim pointer.c
user@user-VirtualBox:~$ gcc -g pointer.c -o pointer
user@user-VirtualBox:~$ ./pointer
=== 변수 정보 ===
num의 값: 10
num의 주소: 0x7ffe41a95cec

=== 포인터 정보 ===
ptr이 저장한 값(주소): 0x7ffe41a95cec
ptr 자신의 주소: 0x7ffe41a95cf0
ptr이 가리키는 값: 10

=== 값 변경 후 ===
num의 값: 50
*ptr의 값: 50

=== 크기 확인 ===
int 크기: 4 byte
포인터 크기: 8 byte
user@user-VirtualBox:~$
```

코드와 실행결과는 위 사진과 같다. ptr는 num을 가리키는 포인터이므로, num값을 변경하면 *ptr값도 바뀌는 것을 확인할 수 있다. 또한, sizeof(ptr)을 하면 8byte가 나오는 것을 확인할 수 있는데, 64비트 운영체제에서는 주소값 크기가 8바이트이기 때문이라는 것을 알 수 있다. 이는 포인터가 가리키는 대상이 char(1바이트)든 int(4바이트)든 상관없이, 항상 8바이트인 것을 의미한다.

예시로, 32bit 주소체계는 16진수로 표현 시 0x12345678이며, 4바이트(8글자) 이고

64bit는 16진수로 표현 시 0x0000007FF7AABBCC이며, 8바이트(16글자)인 것이다.

Challenge 2. 포인터 연산 실습

```
user@user-VirtualBox: ~  
#include <stdio.h>  
  
int main() {  
  
    int arr[3] = {10, 20, 30};  
  
    // 배열의 시작 주소를 저장  
    int *ptr = arr;  
  
    printf("=== 배열 접근 비교 ===\n");  
  
    for(int i = 0; i < 3; i++) {  
  
        printf("arr[%d] = %d\t", i, arr[i]);  
  
        printf("(ptr + %d) = %d\n", i, *(ptr + i));  
    }  
  
    printf("\n=== 주소 확인 ===\n");  
  
    for(int i = 0; i < 3; i++) {  
  
        printf("&arr[%d] = %p\t", i, &arr[i]);  
1,1
```

```
printf("\n=== 포인터 증감 확인 ===\n");  
  
printf("초기 ptr 주소: %p\n", ptr);  
  
ptr++;  
printf("ptr++ 후 주소: %p\n", ptr);  
  
ptr++;  
printf("한 번 더 ptr++ 후 주소: %p\n", ptr);  
  
ptr--;  
printf("ptr-- 후 주소: %p\n", ptr);  
  
return 0;
```

```
user@user-VirtualBox: ~  
user@user-VirtualBox:~$ vim ptrarthiop.c  
user@user-VirtualBox:~$ gcc -g ptrarthiop.c -o ptrarthiop  
user@user-VirtualBox:~$ ./ptrarthiop  
=== 배열 접근 비교 ===  
arr[0] = 10      *(ptr + 0) = 10  
arr[1] = 20      *(ptr + 1) = 20  
arr[2] = 30      *(ptr + 2) = 30  
  
=== 주소 확인 ===  
&arr[0] = 0x7ffd38e3410c      (ptr + 0) = 0x7ffd38e3410c  
&arr[1] = 0x7ffd38e34110      (ptr + 1) = 0x7ffd38e34110  
&arr[2] = 0x7ffd38e34114      (ptr + 2) = 0x7ffd38e34114  
  
=== 포인터 증감 확인 ===  
초기 ptr 주소: 0x7ffd38e3410c  
ptr++ 후 주소: 0x7ffd38e34110  
한 번 더 ptr++ 후 주소: 0x7ffd38e34114  
ptr-- 후 주소: 0x7ffd38e34110  
user@user-VirtualBox:~$
```

코드와 실행결과는 위 사진과 같다. 배열과 포인터 방식을 사용해서 동일한 결과를 출력할 수 있었다. 이를 통해, `arr[i]`는 내부적으로 `*(ptr + i)`와 동일하게 동작한다는 것을 알 수 있다. 즉, 배열 이름 `arr` 자체가 첫 번째 원소의 주소 역할을 하는 것이다.

C언어 설계 원칙상, 배열의 이름은 그 배열이 시작되는 지점의 메모리 주소값으로 해석한다.

예를 들어, `int arr[3];` 라고 선언하면, 컴퓨터는 메모리 어딘가에 3칸을 잡는다. 이때 `arr`이라는 글자 자체는 `&arr[0]` (첫 번째 칸의 주소)와 동일한 의미를 갖게 된다. 포인터 역시 주소값을 담는 변수이므로, 둘의 본질(주소값을 다룸)이 같아지는 것이다.

둘의 차이점은 포인터는 변수라서 자신이 가리키는 주소를 바꿀 수 있지만, 배열은 상수이기 때문에 주소를 바꿀 수 없다는 것이다.

또한 포인터 증감연산 `ptr++`, `ptr--` 는 주소가 1씩 증감하는 것이 아니라, 4byte씩 증감하는 것을 나타낸다. 이는, `ptr`가 `int *`형으로 선언되었기 때문이다. 포인터 연산의 증가량은 포인터가 가리키고 있는 자료형의 크기에 의해 결정된다.

```
int arr[3] = {10, 20, 30};
```

```
int *ptr = arr;
```

여기서 `ptr++`가 4바이트씩 움직이는 이유는

`arr`이 `int` 배열이라서가 아니라, `arr`의 주소를 담고 있는 `ptr` 변수의 타입이 `int *`이기 때문이다.

똑같은 `int arr[3]`을 가리키더라도, 만약 강제로 타입을 바꿔서 선언하면 이동 주소 거리가 달라진다.

```
int arr[3] = {10, 20, 30};
```

```
char *ptr2 = (char *)arr; // int 배열을 1바이트 단위로 보겠다고 선언
```

```
ptr2++; // 이때는 4바이트가 아니라 1바이트만 이동
```

위 코드에서 `ptr`주소가 `3410c -> 34110 -> 34114` 그리고 `ptr--`를 하면 `34110`으로 바뀌는 것을 보면 `ptr`변수의 타입이 `int`이기 때문에 주소값이 4바이트씩 증감된다는 것을 알 수 있다.

Challenge3. call by value vs call by reference 비교

```
#include <stdio.h>

// call by value
void swap_by_value(int a, int b) {
    int temp = a;
    a = b;
    b = temp;

    printf("[swap_by_value 내부] a=%d b=%d\n", a, b);
}

// call by reference
void swap_by_reference(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;

    printf("[swap_by_reference 내부] a=%d b=%d\n", *a, *b);
}

int main() {
    int x = 10;
    int y = 20;
```

```
int main() {

    int x = 10;
    int y = 20;

    printf("=== call by value ===\n");
    printf("호출 전: x=%d y=%d\n", x, y);
    swap_by_value(x, y);
    printf("호출 후: x=%d y=%d\n", x, y);

    printf("\n=== call by reference ===\n");
    printf("호출 전: x=%d y=%d\n", x, y);
    swap_by_reference(&x, &y);
    printf("호출 후: x=%d y=%d\n", x, y);

    return 0;
```

```
user@user-VirtualBox:~$ vim callby.c
user@user-VirtualBox:~$ gcc -g callby.c -o callby
user@user-VirtualBox:~$ ./callby
=== call by value ===
호출 전: x=10 y=20
[swap_by_value 내부] a=20 b=10
호출 후: x=10 y=20

=== call by reference ===
호출 전: x=10 y=20
[swap_by_reference 내부] a=20 b=10
호출 후: x=20 y=10
user@user-VirtualBox:~$
```

코드와 실행결과는 위 사진과 같다.

call by value는 이름 그대로, 값만 복사해서 전달하는 것을 말한다. 위 두번째 사진에서 첫 번째 빨간 동그라미가 그것이다.

```
swap_by_value(x, y);
```

호출 결과를 보면, swap_by_value가 끝나고 복귀하면 원본함수의 x,y값은 변하지 않고 그대로인 것을 알 수 있다. 따라서 함수 내부에서 값을 바꿔도 원본이 유지된다는 특징이 있기 때문에 원본을 보호해야 할 때, 단순 계산용 함수 구현할 때에 필요할 것이라는 것을 알 수 있다.

call by reference는 위와는 다르게, 주소를 전달하는 것을 말한다. 위 두번째 사진에서 두 번째 빨간 동그라미가 그것이다.

```
swap_by_reference(&x, &y);
```

call by value와는 다르게, 인자로 주소를 전달하고 main함수 윗부분 함수에서도 void swap_by_reference(int *a, int *b) 이렇게 *를 붙여서 받는 것을 알 수 있다. 이에 int *a=&x라는 식이 성립되며, a가 포인터의 역할을 하게 된다는 것을 알 수 있다. 따라서 원본을 직접 변경할 수 있다. 위 마지막 사진을 보면, swap_by_reference를 하고 원본인 x, y값도 20, 10으로 바뀌게 되는 것을 볼 수 있다. 따라서 call by reference는 여러 값을 함수에서 수정해야 할 때, 주소값만 전달하므로 큰 데이터의 복사를 줄이고 싶을 때 필요하다라는 것을 알 수 있다.